

Denotational Semantics of Gradual Typing using Synthetic Guarded Domain Theory

PLS Talk

Eric Giovannini Tingting Ding Max S. New

University of Michigan

November 2024

Section 1

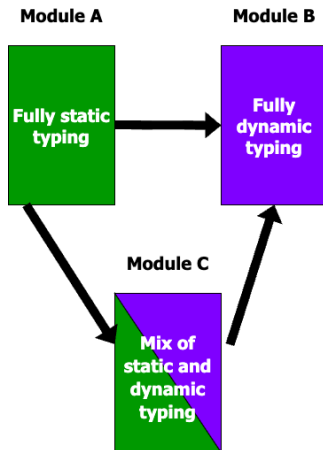
Introduction and Background

Big Idea

This work is about constructing a **denotational semantics** that can validate **relational properties** of **gradually-typed programming languages**

Gradual Typing: What it Is

Idea: Combine static and dynamic typing disciplines in the same language to obtain the benefits of both styles

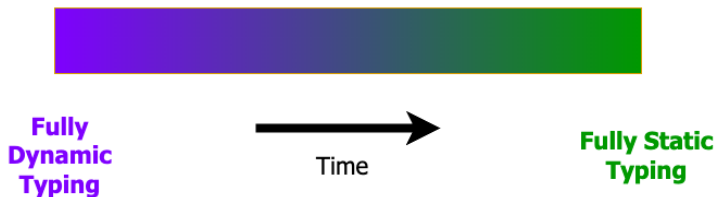


Gradual Typing: Goals

What do we want from a gradually-typed language?

- (“*Gradual*”) Allow the programmer to *gradually* migrate code from a dynamic to static style without changing the meaning of the program

Codebase



- (“*Typing*”) Support type-based reasoning principles for the statically-typed portions of the codebase

Gradual Typing: How it Works

- Gradually-typed languages are typically elaborated to a *cast calculus*

Surface Language $\rightarrow$ Cast Calculus

Elaboration inserts explicit *type casts* at the boundaries between statically and dynamically typed code to ensure that the types are enforced at runtime.

Surface language:

```
def foo(x):
  ...

def add1(n : int):
  return n + 1

x : int = ...
y = foo(x)
z = add1(y)
```

Cast calculus:

```
...
...
...
...
...

x : int = ...
y = foo(cast(int, ?, x))
z = add1(cast(?, int, y))
```

Cast Calculi

A **dynamic type** ? (pronounced “dyn”) assigned to dynamically typed terms

- Stands for a statically unknown type

Type precision: ordering on types from “least dynamic/most precise” to “most dynamic/least precise”:

$$A_1 \sqsubseteq A_2 \text{ (} A_1 \text{ is more precise than } A_2 \text{)}$$

- Fact: $A \sqsubseteq ?$ for all A (? is the most dynamic)

Term Precision

Type precision extends to an ordering on terms, called **term precision**:

$$M_1 \sqsubseteq M_2 \text{ (} M_1 \text{ is more precise than } M_2 \text{)}$$

Intuition: The types in M_1 are more precise than those in M_2 in the type precision ordering

Casts

For any types $A_1 \sqsubseteq A_2$, there is an upcast $\langle A_2 \nwarrow A_1 \rangle$ and a downcast $\langle A_1 \swarrow A_2 \rangle$.

- The upcast always succeeds, but the downcast may fail (error) at runtime.
- There is a term $\mathcal{U}$ (pronounced “error”) representing runtime error, such that $\mathcal{U} \sqsubseteq M$ for all M .

Formal Properties of Gradually Typed Languages

- **Soundness of the equational theory:** formalizes the idea that the statically typed portions of the codebase should enjoy type-based reasoning principles.
 - justifies program optimizations

Formal Properties of Gradually Typed Languages

- **Soundness of the equational theory:** formalizes the idea that the statically typed portions of the codebase should enjoy type-based reasoning principles.
 - justifies program optimizations
- **Graduality:** formalizes the idea that migrating from a static to a dynamic typing discipline should not change the semantics of the program except to catch more errors (Siek et al. 2015; New and Ahmed 2018).
 - e.g., suppose our language has base type Nat
 - Graduality says that if $M \sqsubseteq N : \text{Nat}$, then either (1) M errors, or (2) M and N both diverge, or (3) M and N both evaluate to the same natural number n .

Formal Properties of Gradually Typed Languages

- **Soundness of the equational theory:** formalizes the idea that the statically typed portions of the codebase should enjoy type-based reasoning principles.
 - justifies program optimizations
- **Graduality:** formalizes the idea that migrating from a static to a dynamic typing discipline should not change the semantics of the program except to catch more errors (Siek et al. 2015; New and Ahmed 2018).
 - e.g., suppose our language has base type Nat
 - Graduality says that if $M \sqsubseteq N : \text{Nat}$, then either (1) M errors, or (2) M and N both diverge, or (3) M and N both evaluate to the same natural number n .

A semantics for a gradually-typed language should validate both of these principles.

Our Goal in This Work

Provide an *expressive, reusable, compositional* semantic framework for defining well-behaved semantics of gradually typed programs.

Semantics of Gradually-Typed Languages

- Even simple gradually-typed languages are nontrivial to model!
 - Modeling the dynamic type requires solving non-well-founded recursive equations → **domain theory**

Semantics of Gradually-Typed Languages

- Even simple gradually-typed languages are nontrivial to model!
 - Modeling the dynamic type requires solving non-well-founded recursive equations → **domain theory**
 - Cast calculi involve effects including divergence and errors → **monads** or **adjunctions**

Semantics of Gradually-Typed Languages

- Even simple gradually-typed languages are nontrivial to model!
 - Modeling the dynamic type requires solving non-well-founded recursive equations → **domain theory**
 - Cast calculi involve effects including divergence and errors → **monads** or **adjunctions**
 - Ordering on types and terms → **reflexive graph categories** or **double categories**

Prior Approaches

Denotational models using classical domain theory (New and Licata 2018)

- Benefit: compositional and reusable
- Limitation: can't scale to advanced, highly self-referential language features such as higher-order store (Birkedal, Støvring, and Thamsborg 2009)

Prior Approaches

Denotational models using classical domain theory (New and Licata 2018)

- Benefit: compositional and reusable
- Limitation: can't scale to advanced, highly self-referential language features such as higher-order store (Birkedal, Støvring, and Thamsborg 2009)

Operational step-indexed logical relations (New and Ahmed 2018; New, Licata, and Ahmed 2019)

- Benefit: handles arbitrary recursive definitions
- Limitation: tedious proofs; difficult to reuse results across different languages

Revisting Our Goal

Provide an *expressive, reusable, compositional* semantic framework for defining well-behaved semantics of gradually typed programs.

- Reusable and compositional $\rightsquigarrow$ denotational semantics
- Expressive $\rightsquigarrow$ some form of step indexing

Guarded Domain Theory

- Alternative to classical domain theory that can scale to arbitrarily complex definitions
- Based on an entirely different foundations, most commonly “step-indexed sets”, i.e., objects in the *topos of trees* (Birkedal et al. 2011).
- Idea: infinite sequence of increasingly precise approximations to the true type being modeled.
- Step-indexing can be seen as an *analytic* form of guarded domain theory.

Guarded Domain Theory, Continued

- Operator $\triangleright$ (pronounced “later”) that delays the approximation by one step
- Any guarded domain equation $X \cong F(\triangleright X)$ has a unique solution.
- Main difference from classical domain theory: *intensional* nature — unfolding a guarded domain equation requires an observable “step”

Guarded Type Theory

- Synthetic presentation of guarded domain theory
- $\triangleright$ is taken as a primitive operation on types
- Axiom that *guarded* domain equations have a (necessarily unique) solution.
- Benefit: we model an object language type as a *set* instead of a step-indexed set

Our Goal, Refined

- **Motivating goal:** Adapt the denotational semantics for gradual typing based on classical domain theory to the guarded setting.
- Start off with a simple language, and expand to handle advanced features in future work

Contributions

- We define a denotational semantics for a simple gradually-typed cast calculus in guarded type theory that validates graduality and the soundness of the equational theory.
- Most of our core theorems have been verified in Guarded Cubical Agda.

Section 2

Syntax

Types and Terms

Types A	$::=$	$\text{Nat} \mid ? \mid A \multimap A' \mid A \times A'$
Type Precision c	$::=$	$r(A) \mid cc' \mid c \multimap c' \mid c \times c'$ $\mid \text{Inj}_{\mathbb{N}} \mid \text{Inj}_{\times} \mid \text{Inj}_{\multimap}$
Values V	$::=$	$x \mid \text{up } c \ V \mid \text{zro} \mid \text{suc } V \mid \lambda x.M \mid (V, V')$
Terms M, N	$::=$	$\mathcal{U} \mid \text{up } c \ M \mid \text{dn } c \ M \mid \text{zro} \mid \text{suc } M \mid \lambda x.M$ $\mid MN \mid (M, N) \mid \text{let } (x, y) = M \text{ in } N$
Contexts Γ	$::=$	$\cdot \mid \Gamma, x : A$
Ctx Precision Δ	$::=$	$\cdot \mid \Delta, x : c$

Figure 1: GTLC Cast Calculus Syntax

Type Precision Derivations

- Intuition: A type precision derivation $c : A \sqsubseteq A'$ is a proof that A is more precise than A'
- $c : A \sqsubseteq A'$ corresponds to a relation between A and A' along with a pair of coercions in both directions: the upcast $\text{up } c : A \rightarrow A'$ and the downcast $\text{dn } c : A' \rightarrow A$.

Typing Rules (Selected)

$$\begin{array}{c}
 \Gamma \vdash \mathcal{U} : A \\
 \\
 \frac{\Gamma \vdash M : A \quad c : A \sqsubseteq A'}{\Gamma \vdash \text{up } c \, M : A'} \qquad \frac{\Gamma \vdash N : A' \quad c : A \sqsubseteq A'}{\Gamma \vdash \text{dn } c \, N : A}
 \end{array}$$

Figure 2: GTLC Typing Rules (Selected)

Type Precision Rules

$$\begin{array}{c}
 r(A) : A \sqsubseteq A \\
 \\
 \frac{c : A \sqsubseteq A' \quad c' : A' \sqsubseteq A''}{cc' : A \sqsubseteq A''} \\
 \\
 \text{Inj}_{\rightarrow} : ? \rightarrow ? \sqsubseteq ? \qquad \text{Inj}_{\mathbb{N}} : \text{Nat} \sqsubseteq ? \qquad \text{Inj}_{\times} : ? \times ? \sqsubseteq ? \\
 \\
 \frac{c_i : A_i \sqsubseteq A'_i \quad c_o : A_o \sqsubseteq A'_o}{c_i \rightarrow c_o : (A_i \rightarrow A_o) \sqsubseteq (A'_i \rightarrow A'_o)} \qquad \frac{c_1 : A_1 \sqsubseteq A'_1 \quad c_2 : A_2 \sqsubseteq A'_2}{c_1 \times c_2 : (A_1 \times A_2) \sqsubseteq (A'_1 \times A'_2)}
 \end{array}$$

Figure 3: GTLC Type Precision Derivations

Type Precision Equivalence

- Equational theory $c \equiv c'$ for type precision derivations
- Motivation: equivalent derivations $c, c' : A \sqsubseteq A'$ are not necessarily *equal* in the semantics, because we explicitly account for the steps taken by programs

$$r(A)c \equiv c \qquad c \equiv cr(A') \qquad c(c'c'') \equiv (cc')c''$$

$$r(A_i \multimap A_o) \equiv r(A_i) \multimap r(A_o) \qquad r(A_1 \times A_2) \equiv r(A_1) \times r(A_2)$$

$$(c_i \multimap c_o)(c'_i \multimap c'_o) \equiv (c_i c'_i \multimap c_o c'_o) \qquad (c_1 \times c_2)(c'_1 \times c'_2) \equiv (c_1 c'_1 \times c_2 c'_2)$$

Figure 4: GTLC Type Precision Equivalence

Term Precision Rules

- Extension of type precision to terms
- $\Delta \vdash M \sqsubseteq M' : c$ where Δ is a context where variables are assigned to type precision derivations.
- Not shown: congruence rules for each type constructor, e.g., if $M \sqsubseteq M'$ and $N \sqsubseteq N'$ then $M N \sqsubseteq M' N'$.

$$\frac{\Delta \vdash M \sqsubseteq M' : c \quad c \equiv c'}{\Delta \vdash M \sqsubseteq M' : c'} \text{EQUIVTYPEPREC}$$

$$\Delta \vdash \mathcal{U} \sqsubseteq M : c \text{ERRBOT}$$

$$\frac{M \sqsubseteq M' : c_r}{\text{up } c \ M \sqsubseteq M' : c_r} \text{UPL}$$

$$\frac{M \sqsubseteq M' : c_l}{M \sqsubseteq \text{up } c \ M' : c_l} \text{UPR}$$

The Four Cast Rules

The rules UPL and UPR say that the upcast is a *least upper bound* and dually the rules DNL and DNR say that the downcast is a *greatest lower bound*.

- i.e. for upcasts, UPR says that the upcast is less precise than the original term, but UPL limits the extent to which it is less precise

$$\frac{M \sqsubseteq M' : c c_r}{\text{up } c \ M \sqsubseteq M' : c_r} \text{ UPL}$$

$$\frac{M \sqsubseteq M' : c_l}{M \sqsubseteq \text{up } c \ M' : c_l c} \text{ UPR}$$

Section 3

A Simple Term Model in Guarded Type Theory

More about Guarded Type Theory

- Recall: Operator $\triangleright : \text{Type} \rightarrow \text{Type}$
- $\text{next} : A \rightarrow \triangleright A$ for all A
- *Guarded* fixpoint operator $\text{fix} : (\triangleright A \rightarrow A) \rightarrow A$ satisfying $\text{fix } f = f(\text{next}(\text{fix } f))$
 - Constructing a Prop in this manner corresponds to Löb-induction

Ticked Cubical Type Theory

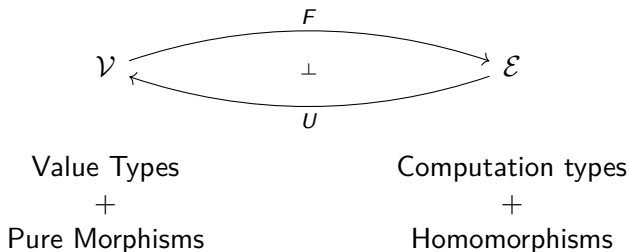
- Abstract sort tick
- $\triangleright A$ is modeled as the Pi-type tick $\rightarrow A$
- The type A is allowed to depend on t ; we write $\triangleright_t A$
- Rules are similar to those of ordinary Π types
- Notation: M_t for tick application

A CBPV Model

- Goal of this section: define a model for only the **types and terms** of GTLC

A CBPV Model

- Goal of this section: define a model for only the **types and terms** of GTLC
- We follow the structure of Levy's Call-by-Push-Value (Levy 1999)
 - Refinement of Moggi's monadic semantics (Moggi 1991) that decomposes Moggi's monad T into an adjunction $T = UF$:



Value and Computation Categories

- Value category: sets and functions
- Computation category: sets equipped with algebraic structure, and homomorphisms that preserve this structure
- Effects we model: errors and computational steps

Value and Computation Categories

- Value category: sets and functions
- Computation category: sets equipped with algebraic structure, and homomorphisms that preserve this structure
- Effects we model: errors and computational steps

Definition (Simple Error Domains)

A (simple) error domain B consists of

- 1 A carrier set UB
- 2 An element $\mathcal{U}_B : UB$ representing error
- 3 A function $\theta_B : \triangleright UB \rightarrow UB$ modeling a computational step

The Free Error Domain

Definition (Free error domain)

For a set A , we define the (carrier of) *free error domain* $U(L_{\mathcal{U}}A)$ as the unique solution to the guarded domain equation:

$$U(L_{\mathcal{U}}A) \cong A + 1 + \triangleright U(L_{\mathcal{U}}A).$$

We use the following notation for the three constructors:

- ① $\eta: A \rightarrow U(L_{\mathcal{U}}A)$
- ② $\mathcal{U}: U(L_{\mathcal{U}}A)$
- ③ $\theta: \triangleright U(L_{\mathcal{U}}A) \rightarrow U(L_{\mathcal{U}}A)$

The Free Error Domain

Definition (Free error domain)

For a set A , we define the (carrier of) *free error domain* $U(L_{\mathcal{U}}A)$ as the unique solution to the guarded domain equation:

$$U(L_{\mathcal{U}}A) \cong A + 1 + \triangleright U(L_{\mathcal{U}}A).$$

We use the following notation for the three constructors:

- ① $\eta: A \rightarrow U(L_{\mathcal{U}}A)$
- ② $\mathcal{U}: U(L_{\mathcal{U}}A)$
- ③ $\theta: \triangleright U(L_{\mathcal{U}}A) \rightarrow U(L_{\mathcal{U}}A)$

- $\mathcal{U}$ and θ provide the error domain structure for $L_{\mathcal{U}}A$
- Notation: We write $\theta_t(\dots)$ to mean $\theta(\lambda t. \dots)$.
- We define $\delta := \theta \circ \text{next} : U(L_{\mathcal{U}}A) \rightarrow U(L_{\mathcal{U}}A)$

Universal Property of the Free Error Domain

- Any function $f : U(L_{\mathcal{U}}A) \rightarrow UB$ uniquely extends to a homomorphism $f^{\dagger} : L_{\mathcal{U}}A \multimap B$ satisfying $Uf^{\dagger} \circ \eta = f$.

Modeling the Type Structure

- Nat denotes the set of natural numbers
- $\times$ denotes the cartesian product of sets.
- CBV function type $A \multimap A'$ denotes the CBPV decomposition
 $U(\llbracket A \rrbracket \rightarrow L_U \llbracket A' \rrbracket)$
 - $A \rightarrow B$ is the functions from A to UB with the obvious point-wise algebraic structure

Modeling the Dynamic Type

Classical domain theory:

$$D \cong \text{Nat} + (D \times D) + U(D \rightarrow L_{\mathcal{U}} D)$$

- No inductive or coinductive solutions

Modeling the Dynamic Type

Classical domain theory:

$$D \cong \text{Nat} + (D \times D) + U(D \rightarrow L_{\mathcal{U}}D)$$

- No inductive or coinductive solutions

Guarded domain theory:

$$D \cong \text{Nat} + (D \times D) + \triangleright U(D \rightarrow L_{\mathcal{U}}D). \quad (1)$$

Modeling the Dynamic Type

Classical domain theory:

$$D \cong \text{Nat} + (D \times D) + U(D \rightarrow L_{\mathcal{U}}D)$$

- No inductive or coinductive solutions

Guarded domain theory:

$$D \cong \text{Nat} + (D \times D) + \triangleright U(D \rightarrow L_{\mathcal{U}}D). \quad (1)$$

- We solve this equation by a mixture of inductive types and guarded fixed points (see paper for details).
- We refer to the three injections for numbers, pairs and functions as $\text{Inj}_{\mathbb{N}}$, $\text{Inj}_{\times}$, and $\text{Inj}_{\rightarrow}$ respectively.

Interpreting the Terms

- Terms $x : A_1, \dots \vdash M : A$ are effectful functions
 $\llbracket M \rrbracket : \llbracket A_1 \rrbracket \times \dots \rightarrow UL_{\cup} \llbracket A \rrbracket$
- Values $x : A_1, \dots \vdash V : A$ are pure functions $\llbracket V \rrbracket : \llbracket A_1 \rrbracket \times \dots \rightarrow \llbracket A \rrbracket$.
- Only nontrivial terms: **casts**

Interpreting Casts

- Upcasts $\rightsquigarrow$ pure functions
 - $\llbracket \text{up } c \rrbracket : \llbracket A \rrbracket \rightarrow \llbracket A' \rrbracket$
- Downcasts $\rightsquigarrow$ homomorphisms of error domains
 - $\llbracket \text{dn } c \rrbracket : L_{\mathcal{U}} \llbracket A' \rrbracket \multimap L_{\mathcal{U}} \llbracket A \rrbracket$.

Interpreting Casts

- Upcasts $\rightsquigarrow$ pure functions
 - $\llbracket \text{up } c \rrbracket : \llbracket A \rrbracket \rightarrow \llbracket A' \rrbracket$
- Downcasts $\rightsquigarrow$ homomorphisms of error domains
 - $\llbracket \text{dn } c \rrbracket : L_{\mathcal{U}} \llbracket A' \rrbracket \multimap L_{\mathcal{U}} \llbracket A \rrbracket$.
- Reflexivity casts $\rightsquigarrow$ identities
 - $\llbracket \text{up } r(A) \rrbracket = \text{id}_{\llbracket A \rrbracket}$
 - $\llbracket \text{dn } r(A) \rrbracket = \text{id}_{L_{\mathcal{U}} \llbracket A \rrbracket}$

Interpreting Casts

- Upcasts $\rightsquigarrow$ pure functions
 - $\llbracket \text{up } c \rrbracket : \llbracket A \rrbracket \rightarrow \llbracket A' \rrbracket$
- Downcasts $\rightsquigarrow$ homomorphisms of error domains
 - $\llbracket \text{dn } c \rrbracket : L_{\mathcal{U}} \llbracket A' \rrbracket \multimap L_{\mathcal{U}} \llbracket A \rrbracket$.
- Reflexivity casts $\rightsquigarrow$ identities
 - $\llbracket \text{up } r(A) \rrbracket = \text{id}_{\llbracket A \rrbracket}$
 - $\llbracket \text{dn } r(A) \rrbracket = \text{id}_{L_{\mathcal{U}} \llbracket A \rrbracket}$
- Transitivity of type precision $\rightsquigarrow$ composition
 - $\llbracket \text{up } (cc') \rrbracket = \llbracket \text{up } c' \rrbracket \circ \llbracket \text{up } c \rrbracket$
 - $\llbracket \text{dn } (cc') \rrbracket = \llbracket \text{dn } c \rrbracket \circ \llbracket \text{dn } c' \rrbracket$

Interpreting Casts

- Upcasts $\rightsquigarrow$ pure functions
 - $\llbracket \text{up } c \rrbracket : \llbracket A \rrbracket \rightarrow \llbracket A' \rrbracket$
- Downcasts $\rightsquigarrow$ homomorphisms of error domains
 - $\llbracket \text{dn } c \rrbracket : L_{\mathcal{U}} \llbracket A' \rrbracket \multimap L_{\mathcal{U}} \llbracket A \rrbracket$.
- Reflexivity casts $\rightsquigarrow$ identities
 - $\llbracket \text{up } r(A) \rrbracket = \text{id}_{\llbracket A \rrbracket}$
 - $\llbracket \text{dn } r(A) \rrbracket = \text{id}_{L_{\mathcal{U}} \llbracket A \rrbracket}$
- Transitivity of type precision $\rightsquigarrow$ composition
 - $\llbracket \text{up } (cc') \rrbracket = \llbracket \text{up } c' \rrbracket \circ \llbracket \text{up } c \rrbracket$
 - $\llbracket \text{dn } (cc') \rrbracket = \llbracket \text{dn } c \rrbracket \circ \llbracket \text{dn } c' \rrbracket$
- Upcasts for products/downcasts for functions $\rightsquigarrow$ functorial actions of $\times$ and $\rightarrow$

Interpreting Casts

- Upcasts $\rightsquigarrow$ pure functions
 - $\llbracket \text{up } c \rrbracket : \llbracket A \rrbracket \rightarrow \llbracket A' \rrbracket$
- Downcasts $\rightsquigarrow$ homomorphisms of error domains
 - $\llbracket \text{dn } c \rrbracket : L_{\mathcal{U}} \llbracket A' \rrbracket \multimap L_{\mathcal{U}} \llbracket A \rrbracket$.
- Reflexivity casts $\rightsquigarrow$ identities
 - $\llbracket \text{up } r(A) \rrbracket = \text{id}_{\llbracket A \rrbracket}$
 - $\llbracket \text{dn } r(A) \rrbracket = \text{id}_{L_{\mathcal{U}} \llbracket A \rrbracket}$
- Transitivity of type precision $\rightsquigarrow$ composition
 - $\llbracket \text{up } (cc') \rrbracket = \llbracket \text{up } c' \rrbracket \circ \llbracket \text{up } c \rrbracket$
 - $\llbracket \text{dn } (cc') \rrbracket = \llbracket \text{dn } c \rrbracket \circ \llbracket \text{dn } c' \rrbracket$
- Upcasts for products/downcasts for functions $\rightsquigarrow$ functorial actions of $\times$ and $\rightarrow$

Interpreting Casts (continued)

- **What about downcasts for products and upcasts for functions?**
- See paper for details.

Casts for the Dynamic Type

- Recall: $D \cong \text{Nat} + (D \times D) + \triangleright U(D \rightarrow L_{\mathcal{U}} D)$.

Casts for the Dynamic Type

- Recall: $D \cong \text{Nat} + (D \times D) + \triangleright U(D \rightarrow L_{\mathcal{U}} D)$.
- Upcasts are just the injections into the coproduct, except for $\text{Inj}_{\rightarrow}$ where we must precompose with next

Casts for the Dynamic Type

- Recall: $D \cong \text{Nat} + (D \times D) + \triangleright U(D \rightarrow L_{\mathcal{U}} D)$.
- Upcasts are just the injections into the coproduct, except for $\text{Inj}_{\rightarrow}$ where we must precompose with next
- Downcasts perform pattern matching and return the value if it is of the correct type; otherwise error
 - In the function case, the function value is only available later
 - So we must insert a θ (i.e., take an observable step) to gain access to the function

Extracting a Well-Behaved Semantics

- Goal: define a partial “big-step semantics” function for closed terms of type `nat`

$$- \Downarrow: \{M \mid \cdot \vdash M : \text{Nat}\} \rightarrow \mathbb{N} + \mathcal{U}$$

Extracting a Well-Behaved Semantics

- Goal: define a partial “big-step semantics” function for closed terms of type `nat`

$$- \Downarrow: \{M \mid \cdot \vdash M : \text{Nat}\} \rightarrow \mathbb{N} + \mathcal{U}$$

- **Clock quantification:** method of expressing definitions in ordinary set-theoretic foundations internally to guarded type theory
- Idea: All guarded constructs are indexed by a clock k , and universally quantifying over k allows us to encode coinductive types in guarded type theory

Extracting a Well-Behaved Semantics

- Goal: define a partial “big-step semantics” function for closed terms of type `nat`

$$- \Downarrow: \{M \mid \cdot \vdash M : \text{Nat}\} \rightarrow \mathbb{N} + \mathcal{U}$$

- **Clock quantification:** method of expressing definitions in ordinary set-theoretic foundations internally to guarded type theory
- Idea: All guarded constructs are indexed by a clock k , and universally quantifying over k allows us to encode coinductive types in guarded type theory

Extracting a Well-Behaved Semantics (continued)

- Recall the free error domain: $UL_{\mathcal{U}}^k X \cong X + 1 + \triangleright^k U(L_{\mathcal{U}}^k X)$
- We define a global version of the free error domain:
 - $(L_{\mathcal{U}}^{gl} X) := \forall k. (L_{\mathcal{U}}^k X)$ (ignoring the U)
- We can show that $L_{\mathcal{U}}^{gl} X$ is isomorphic to Capretta's *delay monad* (Capretta 2005), the coinductive type generated by $\text{now} : Y \rightarrow \text{Delay}(Y)$ and $\text{later} : \text{Delay}(Y) \rightarrow \text{Delay}(Y)$.

Extracting a Well-Behaved Semantics (continued)

- Recall the free error domain: $UL_U^k X \cong X + 1 + \triangleright^k U(L_U^k X)$
- We define a global version of the free error domain:
 - $(L_U^{gl} X) := \forall k. (L_U^k X)$ (ignoring the U)
- We can show that $L_U^{gl} X$ is isomorphic to Capretta's *delay monad* (Capretta 2005), the coinductive type generated by $\text{now} : Y \rightarrow \text{Delay}(Y)$ and $\text{later} : \text{Delay}(Y) \rightarrow \text{Delay}(Y)$.
- Easy to define a partial function for termination of the delay monad, which when combined with the above isomorphism gives the desired big-step semantics function $- \Downarrow$.
- See paper for more details.

Section 4

Towards a Relational Model

The Goal

Provide a compositional interpretation of *type* and *term precision* such that we can extract the graduality relation from our model.

A Naive Approach

- Idea: enhance our value and computation types to carry a poset structure
- For most type formers the structure is defined functorially
- **Key design choice:** ordering on the free error domain $L_{\cup}A$.
- First attempt: *step-insensitive error ordering*

Step-Insensitive Error Ordering

$$\begin{aligned}
 \bot &\lesssim l \text{ iff } \top \\
 \eta x &\lesssim \eta y \text{ iff } x \sqsubseteq y \\
 \theta \tilde{l} &\lesssim \theta \tilde{l}' \text{ iff } \triangleright_t(\tilde{l}_t \lesssim \tilde{l}'_t) \\
 \theta \tilde{l} &\lesssim \bot \text{ iff } \exists n. \theta \tilde{l} = \delta^n(\bot) \\
 \theta \tilde{l} &\lesssim \eta y \text{ iff } \exists n. \exists x \sqsubseteq y. (\theta \tilde{l} = \delta^n(\eta x)) \\
 \eta x &\lesssim \theta \tilde{l}' \text{ iff } \exists n. \exists y \sqsupseteq x. (\theta \tilde{l}' = \delta^n(\eta y))
 \end{aligned}$$

Figure 6: Step-insensitive error ordering

- $\bot$ is the bottom element.
- If both sides return values, check whether they are related.
- If both sides think, check that they are related one time step later.
- If one side has terminated and the other is still thinking, the thinking side must terminate with a related behavior.

Problem: Lack of Transitivity

This relation is *not* a partial ordering, because it is **not transitive**!

A relation R on $U(L_{\cup}A)$ is *right step-insensitive* if $x R y$ implies $x R \delta y$. Left step-insensitivity is defined analogously.

Theorem

Let R be a binary relation on $U(L_{\cup}A)$. If R satisfies transitivity, θ -congruence and left and right step-insensitivity, then R is the total relation: $\forall x, y. x R y$.

Problem: Lack of Transitivity

This relation is *not* a partial ordering, because it is **not transitive**!

A relation R on $U(L \cup A)$ is *right step-insensitive* if $x R y$ implies $x R \delta y$. Left step-insensitivity is defined analogously.

Theorem

Let R be a binary relation on $U(L \cup A)$. If R satisfies transitivity, θ -congruence and left and right step-insensitivity, then R is the total relation: $\forall x, y. x R y$.

Proof.

See paper. □

Lack of Transitivity (continued)

- $\lesssim$ satisfies θ -congruence and left and right step-insensitivity, so if it is also transitive then it is the total relation.
- But e.g., for the flat poset structure on $\mathbb{N}$, $\eta_0 \lesssim \eta_1$ is false.
- Thus $\lesssim$ is **not transitive**.

Why Transitivity is Necessary

- Q: How to prove graduality compositionally?

Why Transitivity is Necessary

- Q: How to prove graduality compositionally?
- Let's try to adapt the classical domain-theoretic approach to the guarded setting.

Proving Graduality Compositionally

- Interpret types as posets and terms as *monotone functions* into the free error domain
- Interpret a type precision derivation $c : A \multimap A'$ as a *poset relation* between the posets $\llbracket A \rrbracket$ and $\llbracket A' \rrbracket$

Proving Graduality Compositionally

- Interpret types as posets and terms as *monotone functions* into the free error domain
- Interpret a type precision derivation $c : A \multimap A'$ as a *poset relation* between the posets $\llbracket A \rrbracket$ and $\llbracket A' \rrbracket$
- *Downward closed*: if $x' \sqsubseteq_A x$ and $x c y$, then $x' c y$.
- *Upward closed*: if $x c y$ and $y \sqsubseteq_{A'} y'$, then $x c y'$.

Proving Graduality Compositionally

- Interpret types as posets and terms as *monotone functions* into the free error domain
- Interpret a type precision derivation $c : A \multimap A'$ as a *poset relation* between the posets $\llbracket A \rrbracket$ and $\llbracket A' \rrbracket$
- *Downward closed*: if $x' \sqsubseteq_A x$ and $x c y$, then $x' c y$.
- *Upward closed*: if $x c y$ and $y \sqsubseteq_{A'} y'$, then $x c y'$.
- Given a poset A , there is an identity poset relation $r(A)$ given by the poset ordering $\sqsubseteq_A$
 - **It is a poset relation because it's transitive**

Proving Graduality Compositionally

- Interpret types as posets and terms as *monotone functions* into the free error domain
- Interpret a type precision derivation $c : A \multimap A'$ as a *poset relation* between the posets $\llbracket A \rrbracket$ and $\llbracket A' \rrbracket$
- *Downward closed*: if $x' \sqsubseteq_A x$ and $x c y$, then $x' c y$.
- *Upward closed*: if $x c y$ and $y \sqsubseteq_{A'} y'$, then $x c y'$.
- Given a poset A , there is an identity poset relation $r(A)$ given by the poset ordering $\sqsubseteq_A$
 - **It is a poset relation because it's transitive**
- Given two poset relations $c : A_1 \multimap A_2$ and $c' : A_2 \multimap A_3$, we define their composition cc' in the usual manner

Modeling Term Precision

- For closed terms: $\llbracket M \sqsubseteq M' : c \rrbracket$ is defined to be the relation $\llbracket M \rrbracket (UL_{\cup} \llbracket c \rrbracket) \llbracket M' \rrbracket$

Modeling Term Precision

- For closed terms: $\llbracket M \sqsubseteq M' : c \rrbracket$ is defined to be the relation $\llbracket M \rrbracket (UL_{\cup} \llbracket c \rrbracket) \llbracket M' \rrbracket$
- For open terms: need a relationship between the denoted monotone functions
 - We say that $f \sqsubseteq_{c_o}^{c_i} g$ if for all $x : A_i$ and $y : A'_i$ with $x c_i y$, we have $f(x) c_o g(y)$
 - We call this relationship a *square*

Modeling Term Precision

- For closed terms: $\llbracket M \sqsubseteq M' : c \rrbracket$ is defined to be the relation $\llbracket M \rrbracket (UL_{\cup} \llbracket c \rrbracket) \llbracket M' \rrbracket$
- For open terms: need a relationship between the denoted monotone functions
 - We say that $f \sqsubseteq_{c_o}^{c_i} g$ if for all $x : A_i$ and $y : A'_i$ with $x \sqsubset_{c_i} y$, we have $f(x) \sqsubset_{c_o} g(y)$
 - We call this relationship a *square*

$$\begin{array}{ccc}
 A_i & \xrightarrow{\quad c_i \quad} & A'_i \\
 f \downarrow & & \downarrow g \\
 A_o & \xrightarrow{\quad c_o \quad} & A'_o
 \end{array}$$

Squares

- “vertical identity” square $\text{id} \sqsubseteq_c^c \text{id}$ for any c
- “horizontal identity” square $f \sqsubseteq_{r(A_o)}^{r(A_i)} f$ for any monotone function $f : A_i \rightarrow A_o$

Squares

- “vertical identity” square $\text{id} \sqsubseteq_c^c \text{id}$ for any c
- “horizontal identity” square $f \sqsubseteq_{r(A_o)}^{r(A_i)} f$ for any monotone function $f : A_i \rightarrow A_o$

Vertical composition:

If $f \sqsubseteq_{c_2}^{c_1} f'$ and $g \sqsubseteq_{c_3}^{c_2} g'$, then
 $g \circ f \sqsubseteq_{c_3}^{c_1} g' \circ f'$

$$\begin{array}{ccc}
 A_1 & \xrightarrow{c_1} & A'_1 \\
 f \downarrow & & \downarrow f' \\
 A_2 & \xrightarrow{c_2} & A'_2 \\
 g \downarrow & & \downarrow g' \\
 A_3 & \xrightarrow{c_3} & A'_3
 \end{array}$$

Horizontal composition:

If $f \sqsubseteq_{c_o}^{c_i} g$ and $g \sqsubseteq_{c'_o}^{c'_i} h$, then
 $f \sqsubseteq_{c_o c'_o}^{c_i c'_i} h$.

$$\begin{array}{ccccc}
 A_i & \xrightarrow{c_i} & A'_i & \xrightarrow{c'_i} & A''_i \\
 f \downarrow & & g \downarrow & & h \downarrow \\
 A_o & \xrightarrow{c_o} & A'_o & \xrightarrow{c'_o} & A''_o
 \end{array}$$

The Graduality Proof

- Goal: interpret **term precision rules**
- Most cases are just congruence rules and are easy to establish
 - Use composition of squares and functorial actions of the type constructios on squares

The Graduality Proof

- Goal: interpret **term precision rules**
- Most cases are just congruence rules and are easy to establish
 - Use composition of squares and functorial actions of the type constructios on squares
- The core of the proof of graduality: proving the validity of the **cast rules** UPL , UPR , DNL , and DNR .

Representable Relations

- The four cast rules specify a relationship between the semantics of type precision derivations c and the corresponding casts $\text{up } c$, $\text{dn } c$

Representable Relations

- The four cast rules specify a relationship between the semantics of type precision derivations c and the corresponding casts $\text{up } c$, $\text{dn } c$

$$\frac{M \sqsubseteq M' : c c_r}{\text{up } c \, M \sqsubseteq M' : c_r} \text{ UPL}$$

$$\begin{array}{ccccc} \llbracket A_1 \rrbracket & \xrightarrow{\llbracket c \rrbracket} & \llbracket A_2 \rrbracket & \xrightarrow{\llbracket c' \rrbracket} & \llbracket A_3 \rrbracket \\ \downarrow \llbracket \text{up } c \rrbracket & & & & \downarrow \text{id} \\ \llbracket A_2 \rrbracket & \xrightarrow{\llbracket c' \rrbracket} & & & \llbracket A_3 \rrbracket \end{array}$$

Representable Relations

- The four cast rules specify a relationship between the semantics of type precision derivations c and the corresponding casts $\text{up } c$, $\text{dn } c$

$$\frac{M \sqsubseteq M' : c c_r}{\text{up } c \, M \sqsubseteq M' : c_r} \text{ UPL}$$

$$\begin{array}{ccccc} \llbracket A_1 \rrbracket & \xrightarrow{\llbracket c \rrbracket} & \llbracket A_2 \rrbracket & \xrightarrow{\llbracket c' \rrbracket} & \llbracket A_3 \rrbracket \\ \downarrow \llbracket \text{up } c \rrbracket & & & & \downarrow \text{id} \\ \llbracket A_2 \rrbracket & \xrightarrow{\llbracket c' \rrbracket} & & & \llbracket A_3 \rrbracket \end{array}$$

- Problem: this rule quantifies over an arbitrary relation $\llbracket c' \rrbracket$!
- This means the definition of relation is self-referential!

A Simpler Square

Consider this simpler square:

$$\begin{array}{ccc}
 A_1 & \xrightarrow{\quad c \quad} & A_2 \\
 \text{up } c \downarrow & \text{UpL} & \downarrow \text{id} \\
 A_2 & \xrightarrow{\quad r(A_2) \quad} & A_2
 \end{array}$$

- This square, in conjunction with a related one for UPR , say that the relation c is *representable* by the morphism $\text{up } c$

Composing the Squares

Derive the original UPL square from the simpler UPL square:

$$\begin{array}{ccccc}
 A_1 & \xrightarrow{c} & A_2 & \xrightarrow{c'} & A_3 \\
 \text{up } c \downarrow & & \text{id} \downarrow & & \downarrow \text{id} \\
 A_2 & \xrightarrow{r(A_2)} & A_2 & \xrightarrow{c'} & A_3
 \end{array}$$

The composition of $r(A_2)$ with c' is equal to c' , because c' is *downward-closed*

Composing the Squares

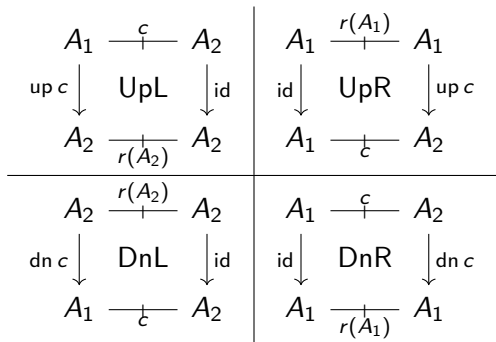
Derive the original UPL square from the simpler UPL square:

$$\begin{array}{ccccc}
 A_1 & \xrightarrow{c} & A_2 & \xrightarrow{c'} & A_3 \\
 \text{up } c \downarrow & & \text{id} \downarrow & & \downarrow \text{id} \\
 A_2 & \xrightarrow{r(A_2)} & A_2 & \xrightarrow{c'} & A_3
 \end{array}$$

The composition of $r(A_2)$ with c' is equal to c' , because c' is *downward-closed*

We can similarly derive the squares for the other three rules by composition.

Simpler Squares



Key Takeaway

We can carry out the graduality proof compositionally, *provided that the relations $\llbracket c \rrbracket$ are closed under the orderings on each side.*

In particular, the ordering relations $r(A)$ must be transitive.

Returning to the Free Error Domain

- Recall: The step-insensitive error ordering is not transitive
- Solution: Give up on step-insensitivity

Lock-Step Error Ordering

Lock-step error ordering $\boxed{l_1 \sqsubseteq l_2}$

$$\eta x \sqsubseteq \eta y \text{ if } x \sqsubseteq_A y$$

$$\mathcal{U} \sqsubseteq l'$$

$$\theta \tilde{l} \sqsubseteq \theta \tilde{l}' \text{ if } \triangleright_t(\tilde{l}_t \sqsubseteq \tilde{l}'_t)$$

- The partial ordering for $UL_{\mathcal{U}}A$ is the lock-step error ordering.
- We similarly define a heterogeneous version of the lock-step ordering that lifts poset relation $c : A \multimap A'$ to a poset relation $L_{\mathcal{U}}c : L_{\mathcal{U}}A \multimap L_{\mathcal{U}}A'$.

Accounting for Steps Taken by Casts

Observation: there are terms related by term precision that take differing numbers of steps.

Consider the DNL square corresponding to the type precision derivation $\text{Inj}_{\rightarrow} : ? \rightarrow ? \sqsubseteq ?$

$$\begin{array}{ccc}
 L_{\cup} D & \xrightarrow[r(L_{\cup} D)]{+} & L_{\cup} D \\
 \downarrow \llbracket \text{dn Inj}_{\rightarrow} \rrbracket & & \downarrow \text{id} \\
 L_{\cup} U(D \rightarrow L_{\cup} D) & \xrightarrow[r(L_{\cup} \llbracket \text{Inj}_{\rightarrow} \rrbracket)]{+} & L_{\cup} D
 \end{array}$$

The RHS always takes zero steps, while the LHS takes a step if the input is a function!

Inserting a Delay

Solution: insert a delay function $\delta^* = (\delta \circ \eta)^\dagger$ on the RHS (recall $\delta : U(L_{\mathcal{U}}A) \rightarrow U(L_{\mathcal{U}}A)$)

$$\begin{array}{ccc}
 L_{\mathcal{U}}D & \xrightarrow{\quad r(L_{\mathcal{U}}D) \quad} & L_{\mathcal{U}}D \\
 \text{\scriptsize $\llbracket \text{dn Inj} \rightarrow \rrbracket$} \downarrow & & \downarrow \delta^* \\
 L_{\mathcal{U}}U(D \rightarrow L_{\mathcal{U}}D) & \xrightarrow{\quad L_{\mathcal{U}}\llbracket \text{Inj} \rightarrow \rrbracket \quad} & L_{\mathcal{U}}D
 \end{array}$$

Now both sides are in lock step!

We call a function that is the identity function except for the introduction of computational steps a **semantic perturbation**.

Weak Bisimilarity on the Free Error Domain

- **Idea:** formalize the property of being equivalent “except for steps”
- Parameterized by a reflexive, symmetric relation on A

Weak bisimilarity $\boxed{l_1 \approx l_2}$

$$\mathcal{U} \approx \mathcal{V} \text{ iff } \top$$

$$\eta x \approx \eta y \text{ iff } x \approx_A y$$

$$\theta \tilde{x} \approx \theta \tilde{y} \text{ iff } \triangleright_t(\tilde{x}_t \approx \tilde{y}_t)$$

$$\theta \tilde{x} \approx \mathcal{U} \text{ iff } \exists n. \theta \tilde{x} = \delta^n(\mathcal{U})$$

$$\theta \tilde{x} \approx \eta y \text{ iff } \exists n. \exists x \approx_A y. (\theta \tilde{x} = \delta^n(\eta x))$$

$$\mathcal{U} \approx \theta \tilde{y} \text{ iff } \exists n. \theta \tilde{y} = \delta^n(\mathcal{U})$$

$$\eta x \approx \theta \tilde{y} \text{ iff } \exists n. \exists y \approx_A x. (\theta \tilde{y} = \delta^n(\eta y))$$

Summary of Relations on the Free Error Domain

Lock-step error ordering

sensitive to steps

transitive

accounts for errors

Weak Bisimilarity

insensitive to steps

not transitive

only relates terms with same stepping behavior

New Interpretation of Term Precision

$\Delta \vdash M \sqsubseteq N : c$ means not that $\llbracket M \rrbracket$ and $\llbracket N \rrbracket$ are necessarily in the lock-step error ordering directly, but that they can be “synchronized” to do so, i.e.:

There exist $f \approx \llbracket M \rrbracket$ and $g \approx \llbracket N \rrbracket$ such that $f \sqsubseteq_{UL_U \llbracket c \rrbracket}^{\llbracket \Delta \rrbracket} g$.

Section 5

Completing the Model

Note

This section gives a high-level overview; much more detail can be found in Section 5 of the paper.

Changes to the Poset Semantics

- Equip all types with a *bisimilarity relation* in addition to an ordering relation
- Morphisms must preserve **both** relations
- Define semantic perturbations as endomorphisms that are weakly bisimilar to the identity morphism

Changes to the Poset Semantics (continued)

- Weaken the notion of representability so that the upcasts and downcasts are not required to be in lock-step with the identity but instead with some *perturbation*.
 - We call this notion *quasi-representability*
- E.g. the modified UPL rule:

$$\begin{array}{ccc}
 A & \xrightarrow{c} & A' \\
 \text{up } c \downarrow & & \downarrow \delta_c^r \\
 A' & \xrightarrow{r(A')} & A'
 \end{array}$$

Finishing the Graduality Proof

- Remainder of the proof: show that quasi-representability is preserved by the constructions on type precision.
- **Problem:** the perturbations are not as well-behaved as actual identity functions
- **Solution:** attach additional data to our types that is associated with semantic perturbations but that we can manipulate syntactically

Syntactic Perturbations

- A syntactic representation of semantic perturbations
- Presented as data associated to each type so that they are amenable to operations such as composition and functorial actions
- Each type is equipped with a monoid of syntactic perturbations and a means of *interpreting* them as semantic perturbations.

Push and Pull

Need to impose an additional condition saying how relations interact with syntactic perturbations:

- We can *push* a perturbation δ_1 on A to a perturbation $\text{push}(\delta_1)$ on A' such that the following square exists:

$$\begin{array}{ccc}
 A & \xrightarrow{c} & A' \\
 \delta_1 \downarrow & & \downarrow \text{push}(\delta_1) \\
 A & \xrightarrow{c} & A'
 \end{array}$$

- Likewise we can *pull* a perturbation on A' to one on A

Last Step: Extensional Squares

- Define a notion of *extensional square* combining bisimilarity with the error ordering:

$$\begin{array}{ccc}
 A_i & \xrightarrow{c_i} & A'_i \\
 f \left(\begin{array}{c} \approx \\ \downarrow \end{array} \right) f' & \sqsubseteq & g' \left(\begin{array}{c} \approx \\ \downarrow \end{array} \right) g \\
 A_o & \xrightarrow{c_o} & A'_o
 \end{array}$$

Relational Adequacy

Show that for closed terms of base type, term precision implies that the the big-step term semantics satisfies the graduality property.

Theorem

If $M \sqsubseteq N : \text{nat}$ then either:

1. $M \Downarrow \mathcal{U}$
2. $M \Downarrow$ and $N \Downarrow$ are both undefined
3. $M \Downarrow n$ and $N \Downarrow n$

where $- \Downarrow$ is the big-step term semantics (a partial function from terms to $\mathbb{N} + \mathcal{U}$)

- See paper for the proof!

Section 6

Discussion and Future Work

Agda Mechanization

- Formalized most of the key pieces of the model construction in Guarded Cubical Agda (Veltri and Vezzosi 2020)
 - Predomain/error domains, morphisms, relations, squares
 - Free error domain
 - Value/computation objects, syntactic perturbations, semantic perturbations, push-pull property
 - Quasi-representable relations
 - The dynamic type as a value object, along with the relations corresponding to the three injections
 - Adequacy of the model with respect to the graduality property

Agda Mechanization

- Formalized most of the key pieces of the model construction in Guarded Cubical Agda (Veltri and Vezzosi 2020)
 - Predomain/error domains, morphisms, relations, squares
 - Free error domain
 - Value/computation objects, syntactic perturbations, semantic perturbations, push-pull property
 - Quasi-representable relations
 - The dynamic type as a value object, along with the relations corresponding to the three injections
 - Adequacy of the model with respect to the graduality property
- Some formalization work remains:
 - Lemmas about quasi-representability of actions of $\rightarrow$ and $\times$ on relations
 - Verifying type precision equivalence rules in the semantics
- <https://github.com/ericgiovannini/gradual-typing-semantics-in-sgdt>

Future Work

- Extend to languages with higher-order store and/or dynamic type-tag generation
- Prove graduality for alternative cast semantics, e.g., eager (Herman, Tomb, and Flanagan 2010) and transient (Vitousek, Swords, and Siek 2017) cast semantics
- Utilize the *intensional* nature of our model to analyze the efficiency of different cast semantics

End

Thank you!

References

- Birkedal, Lars, Rasmus Ejlers Mogelberg, Jan Schwinghammer, and Kristian Støvring. 2011. “First Steps in Synthetic Guarded Domain Theory: Step-Indexing in the Topos of Trees.” In *ACM/IEEE Symposium on Logic in Computer Science (LICS), Toronto, Canada*.
<https://doi.org/10.1109/LICS.2011.16>.
- Birkedal, Lars, Kristian Støvring, and Jacob Thamsborg. 2009. “Realizability Semantics of Parametric Polymorphism, General References, and Recursive Types.” In *Foundations of Software Science and Computational Structures, Berlin*, 456–70.
https://doi.org/10.1007/978-3-642-00596-1/_32.
- Capretta, Venanzio. 2005. “General Recursion via Coinductive Types.” *Logical Methods in Computer Science* Volume 1, Issue 2 (July).
[https://doi.org/10.2168/LMCS-1\(2:1\)2005](https://doi.org/10.2168/LMCS-1(2:1)2005).
- Herman, David, Aaron Tomb, and Cormac Flanagan. 2010. “Safe, Efficient, Gradual Typing.” *ACM Conference on Programming Language Design and Implementation*, 2010, 1–12.
<https://doi.org/10.1145/1809096.1809100>.
- Eric Giovannini, Tingting Ding, Max S. New. 2024. “Denotational Semantics of Gradual Typing.” *Journal of Functional Programming*, 34(1), 1–44.
<https://doi.org/10.1017/S0022253523000137>.